

华南理工大学

探 索 性 实 验

报 告

课程名称：	《数据挖掘》
实验名称	景区游览指南的程序性知识挖掘
组号	第___组
组长姓名/学号	组长姓名/学号
组员姓名/学号	组员 1 姓名/学号 组员 2 姓名/学号
学生专业：	软件工程
授课教师：	蔡毅 / 许可 （写自己班的老师）
开课学期：	2026-2027-01

华南理工大学软件学院

2026 年 10 月

地　　点：	B7 后座 楼　　133/231/233
实验日期与时间：	第 9 周周三 9-12 节 第 11 周周三 9-12 节
总评分	分/100 分
实验项目 1 得分	分/100 分
实验项目 2 得分	分/100 分
实验项目 3 得分	分/100 分
实验项目 4 得分	分/100 分
实验项目 5 得分	分/100 分
实验项目 6 得分	分/100 分
实验项目 7 得分	分/100 分
实验总结反思得分	分/100 分
探索性实验总体批改意见：	
批改人	蔡毅、许可

*请不要修改此页

小组信息和组内评分

组号：第　　组

姓名	角色	主要工作和贡献	组内评分
	组长		%
	组员		%
	组员		%

*说明：1、请提交报告时，组长组织组员一起填写好本页，组内评分请根据实际情况，进行合理分配；2、组内人数为N人，组内评分总和为N*100%

共同要求

允许使用等价工具或技术路线。允许 LLM 辅助，但须解释关键逻辑、规则或 Prompt 及验证方法。截图用于证明运行，不能替代源代码、结构化结果和关键中间产物；报告中的结果应能与代码、文件逐一对应。

章节	核心工作 / 报告重点	评分
第 1 章	3 个景区的数据准备、清洗、结构化与质量检查	基础 100 分；附加后封顶 100 分
第 2 章	三类实体定义、识别、位置记录与可视化迭代	100 分
第 3 章	路线解析、时间层级、名称归一化；路线对比为附加题	基础 100 分；附加后封顶 100 分
第 4 章	代词检测、人工标注、消解与效果评估	100 分
第 5 章	条件—建议抽取、分类去重与统计图	100 分
第 6 章	多源融合、图谱模式、构建、展示与质量验证	100 分
第 7 章	LLM 接入、图谱工具、Agent 闭环与多场景测试	100 分
第 8 章	至少 2 个失败现象；局限性与 1 个突破点的研究方案	100 分

《数据挖掘》探索性实验报告
—实验项目 1 景区指南数据采集与预处理

1.1 实验目标与数据准备

数据自行采集，没有使用课程提供的通用语料。官方路线取自三个景区官网的“行程推荐”栏目，游客游记取自微信公众号。三个景区分成两类：广州长隆与珠海长隆是主题乐园，九寨沟是自然遗产，用来检验同一套实体体系和解析规则能否跨类型复用。

采集脚本在 src/crawler.py，入口配置是 config/sources.json。爬虫用 requests 加 BeautifulSoup，按站点分别写正文容器规则：长隆是 div.c--static-flow，九寨沟取几个 div.g-grid 里文字最多的那个，公众号取 #js_content。每篇抓完立刻存原始 HTML，文件名即 doc_id，后续核对以此为准。

访问策略写死在代码里，方便检查：收到 403/429 就停止本次运行对该主机的后续请求，不轮换会话、不预热首页、不换 IP 去绕过限制；只有瞬时网络错误和服务器 5xx 做退避重试，401/404/410 直接记失败。请求之间保留间隔。

19 个原始页面随工程提交在 data/raw/source_archive/，附一份索引表。重跑不需要联网：python src/rebuild_from_archive.py 从存档重解析出 17 份原始正文，python src/run_all.py 继续跑完后面六个项目。

表 1-1　三个景区的数据来源与规模

景区	类型	官方路线来源	游记来源	官方/游记
广州长隆旅游度假区	主题乐园	官网“行程推荐”3 个页面	微信公众号 5 篇	3 / 5
珠海横琴长隆国际海洋度假区	主题乐园	官网“行程推荐”2 个页面	微信公众号 3 篇	2 / 3
九寨沟风景名胜区	自然遗产	官网“游玩方式”1 个页面	微信公众号 3 篇	1 / 3
合计	2 类	6 个页面	11 篇	6 / 11

最终入库 17 篇（表 1-1）：官方路线 6 篇、游记 11 篇，每篇都是 2000 字以上的长文，清洗后正文合计 46236 字。采集与清洗全程只用公开页面，不含用户个人信息，数据仅用于本次课程实验。

1.2 清洗方法与实现流程

处理流程拆成抽取、清洗、质检三段，一段一个脚本。extractors.py 只负责从 HTML 里取出正文块，clean.py 只按规则处理文本，quality_check.py 只做统计和出图。这样拆的原因是抽取规则随站点变化，清洗规则基本不变，两者混在一个函数里改一处要重测另一处。

清洗规则共 9 条，按 R1 到 R9 编号。前 6 条处理字符级噪声，R7 做全角转半角，R8 删文档内重复行，R9 只标记不删除。每条规则的删除量单独计数，写进 quality_stats.json。

表 1-2　清洗规则与逐条删除量

编号	处理对象	做法	实际删除
R1	零宽字符、控制字符、全角空格	正则剔除后统一半角空格	0 个
R2	残留 HTML 标签与实体	去标签、转义字符还原	0 个（抽取阶段已剥离 15255 个）
R3	行内多空格、首尾空白、连续空行	空白压缩 + strip + 空行合并	逐行归一，未单独计数
R4	模板噪声行	22 条正则整行匹配，命中即删	15 行
R5	网址、邮箱、电话、微信号	正则匹配后删除	网址 1、电话 3，其余 0
R6	emoji、列表符号、重复标点	emoji 删除，列表符号统一为“- ”，重复标点压成一个	emoji 7 个
R7	全角数字字母、时间冒号	全角转半角，个位小时补零（7:00 → 07:00）	逐字符处理
R8	文档内重复行	同一文档内完全相同的行只留一次	50 行
R9	空值、过短、标题缺失、近重复	标记 quality_flags 与 is_near_duplicate，不删除	17 篇全部通过

表 1-2 里 R1 和 R2 的删除量都是 0，不是规则没生效，而是这两类噪声在抽取阶段就被处理掉了：BeautifulSoup 解析时标签不进正文，零宽字符在写入 jsonl 前已过滤。R2 那一行括号里的 15255 是抽取阶段从 15 个页面剥离的标签总数，平均每页 1017 个。

两个选择需要说明。模板噪声按整行删除，不做词级替换：“关注公众号获取更多攻略”整行删掉没有损失；“建议”这类词如果在全文里替换，会把“如果带老人建议走南门”这样的条件句一起改坏。符号则只换不改字：原文用 √、●、★ 列的清单统一换成“- ”，文字不动；箭头、破折号、“【上午】”、“第 X 站（10:00）”这些带顺序和时间信息的符号全部保留，它们是项目 3 的解析依据。

空值、重复、异常字段按三种方式处理：能修的修（全角转半角、时间补零），该删的删（重复行、纯符号碎行），不确定的只标记（标题缺失、发布日期为空、近重复）。标记的字段保留原值，不做填充，避免把推断结果当成原文事实。

1.3 结构化数据设计

结构化输出三个文件，分工不同。documents.csv 只放元数据与统计字段，不含正文，17 行可以一屏看完。documents.json 带 content 和 paragraphs，后续项目直接从这里读。dataset.xlsx 有七个 Sheet：文档清单、正文、段落、字段说明、清洗统计、景区汇总、规则统计。

字段共 22 个，分四组。标识组是 doc_id、scenic_code、scenic_area；来源组是 source_platform、source_url、author、publish_date；内容组收了 title、content、raw_content 三项；剩下的六个属质量组：raw_char_len、clean_char_len、noise_line_removed、dup_line_removed、has_time_expr、is_valid。主要字段的含义见表 1-3。

表 1-3　主要字段定义

字段	类型	说明
doc_id	str	文档唯一编号，格式为“景区码-类型-序号”，如 GZCL-NOTE-03
scenic_code	str	景区码：GZCL 广州长隆、ZHCL 珠海长隆、JZG 九寨沟
text_type	str	official_route 官方路线 / travel_note 游客游记
source_url	str	原始链接，公众号链接带时效签名，过期后以本地 HTML 为准
has_time_expr	int	正文是否含时刻表达，1 有 0 无
content	text	清洗后正文，只在 documents.json 提供（CSV 不含正文）
paragraphs	list	段落列表，每段含 i / start / end；不写进 CSV，见 paragraphs.csv 与「段落」表
is_valid	int	是否通过全部质量检查

doc_id 的命名规则是“景区码-类型-序号”。同一景区同一类型的文档连续编号，排序后能按景区聚合，也能按类型过滤。项目 3 的路线编号在 doc_id 后面追加 -R1、-R2，表示同一篇文档切出的第几条路线。

段落表另存为 paragraphs.csv，列为 doc_id、para_id、char_start、char_end、字数、段落文本，一行一段，17 篇共切出 1096 段。区间左闭右开，content[start:end] 就是该段原文。项目 2 标实体位置时先定段，再用 start 加段内偏移换算全篇位置；项目 3 也是逐段读的。

1.4 清洗前后对比与数据质量

清洗前后共减少 907 字，删除率 1.9%（47143 → 46236）。这个比例偏低，因为抽取阶段已经除掉了页面结构，清洗只需要处理行级噪声。逐条删除量见表 1-2，分景区统计见表 1-4，各景区文档数与清洗前后长度见图 1-1、图 1-2。

表 1-4　分景区清洗统计

景区	篇数	原文均长	清洗后均长	噪声行	重复行
广州长隆	8	3215	3192	3	9
珠海长隆	5	3005	2891	10	37
九寨沟	4	1599	1561	2	4
合计	17	2773	2719	15	50

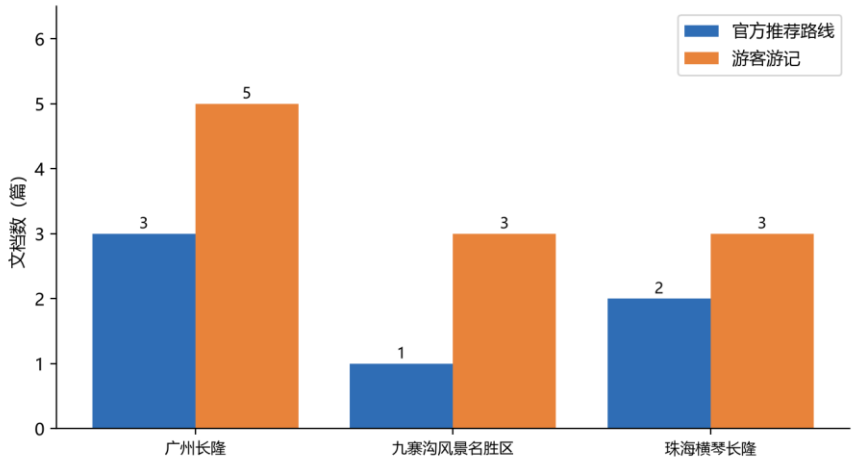


图 1-1　各景区采集文档数量（官方路线 / 游客游记）

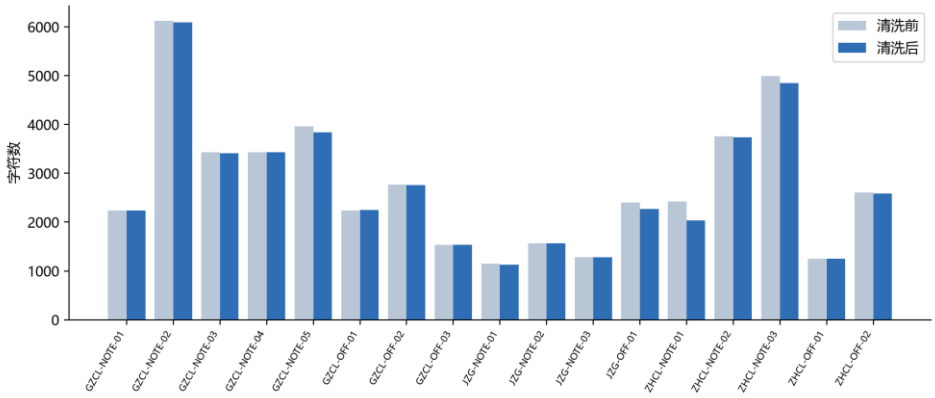


图 1-2　清洗前后正文长度对比

三例可核验的对比。第一例是时间格式：九寨沟官方路线原文写“沟口乘车→原始森林（7:00-8:10 一个小时车程）”，清洗后为“07:00-08:10”，个位小时补零，其余不动。第二例是列表符号：一篇游记用“ √另一半”列随身物品，清洗后改为“- 另一半”，文字保留。第三例是模板噪声：ZHCL-NOTE-01 正文夹了 8 行“关注公众号”类句子和 33 行重复碎行，按 R4 与 R8 删除后正文连贯，字数从 2418 降到 2030。

质量检查做五项：数量、缺失、重复、格式、长度。结果见表 1-5，噪声行与来源平台的分布见图 1-3、图 1-4。三景区各自达到“至少 1 条官方路线 +3 篇游记”的要求。重复一项也没有踩线：17 篇里没有标题重复，近重复按 3-gram Jaccard 0.8 的阈值判断，结果也是 0，没有空文档。长度分布右偏，最短 1131 字、最长 6089 字、中位数 2270 字，长的那几篇来自攻略类公众号。

表 1-5　质量检查结果（Q1 – Q5）

检查项	方法	结果
Q1 数量	按景区统计官方/游记条数	广州 3/5、珠海 2/3、九寨沟 1/3，全部达标
Q2 缺失	逐字段检查空值	publish_date 有 3 篇为空（2 篇正文快照、1 个官网页面未标日期），已标记
Q3 重复	标题完全匹配 + 正文 3-gram Jaccard ≥ 0.8	标题重复 0、近重复 0
Q4 格式	正则检查乱码与控制字符	乱码 0、控制字符 0
Q5 长度	统计清洗后字数	最短 1131、最长 6089、中位 2270、均值 2719.8

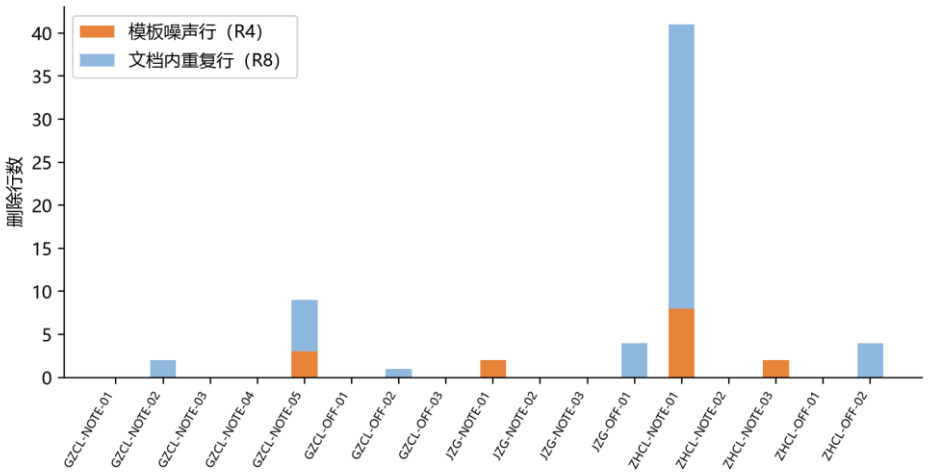


图 1-3　各文档删除的噪声行与重复行

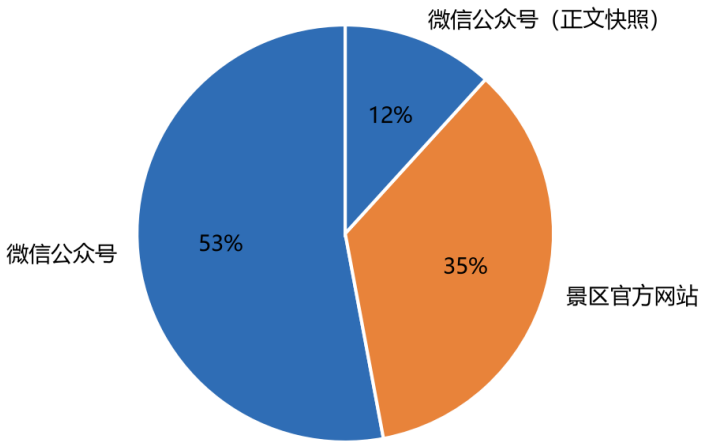


图 1-4　文档来源平台分布

1.5 问题与分析

采集和存档盘点中有四类情况需要说明。存档 19 个页面里只有 15 个进入数据集。另外 4 个的情况是：JZG-NOTE-02、ZHCL-NOTE-04、ZHCL-NOTE-05 三个页面的正文与已收录的页面完全相同，

ZHCL-OFF-03 只提取到 20 个字符、页面没有正文。这 4 个没有删掉，留在存档里说明它们确实访问过。逐条的决定和理由写在 data/raw/archive_inventory.csv。

官网“攻略”栏目多为列表页。三个页面的标题看起来是完整攻略，抓下来正文容器里只有 18 到 45 个字，点开才发现是分类入口。处理办法是先按标题长度和链接层级筛掉列表页，抓完再按正文字数下限（300 字）拦截，不达标的记进 crawl_log.csv 并换下一篇。

搜狗微信检索有 IP 级限流，累计约 30 次请求后开始返回空结果或验证码页。触发后对九寨沟的 2 篇游记改用检索服务返回的正文快照，原文存到 data/raw/manual/，数据集中标注为“微信公众号（正文快照）”。这 2 篇的可复现性弱于其余 15 篇，正文和来源链接都在，但无法通过脚本重跑得到同样结果。

公众号链接带时效签名。source_url 里的 timestamp 与 signature 过期后原链接打不开，所以每篇都留了本地 HTML 快照（data/raw_html/），复核以快照为准。这个限制也意味着数据集不适合作为长期可引用的语料，只能在本次实验范围内使用。

《数据挖掘》探索性实验报告
—实验项目 2 游览步骤的实体识别

2.1 输入数据与实体定义规范

输入是项目 1 的 documents.json：17 篇、46236 字、1096 个段落。识别在段落层面做，再用段落的 start 偏移换算成全篇位置，位置信息能和原文逐字对应。

三类实体的定义与边界见表 2-1。定义在三个景区之间保持一致，区别只在词典内容，判定规则不区分景区。

表 2-1　三类实体的定义、边界与示例

类型	定义	边界（不收录的情况）	示例
景点 POI	可被当作游览目标进入或观看的具体地点	园区、度假区等泛称；城市名；动作短语	五花海、鲸鲨馆、百虎山、南门
交通方式	完成位移所借助的工具或方式	道路名、车站建筑、票种	观光车、小火车、缆车、步行、自驾
时间节点	时刻、时段、时长、天序	与游览无关的营业时间说明中的时刻	07:00、上午、30 分钟、第二天

三类之间有两处容易混。“南门”“北门”既是地点也是出入口，路线里常作为起终点出现，所以按景点处理。“空中缆车”既指设施也指乘坐方式，项目 2 按交通方式收录，项目 3 解析路线时也填进 transport 字段，两处一致。“长隆”单独出现时不收，它可能指度假区、园区或品牌，指代不明。

2.2 识别方法与批量实现

识别用 jieba 分词加自定义词典，时间是正则。词典按景区分开写，识别时合并三份再加上通用词，共 130 余条；另有一份 47 词的停用词表，只在第二版使用。词典匹配用长词优先、占位不重叠的方式：先把词按长度倒序，命中后把这几个字标记为已占用，后面的短词就不能再覆盖同一段文字。这样“长海观景台”不会被“长海”切走，“长隆野生动物世界”也不会被“野生动物世界”重复计数。

词典之外还有一层后缀兜底：jieba 切出来、以地点后缀（园、馆、沟、山、湖、瀑、寨等）结尾的词，补进来当景点。这一层能覆盖词典没收录的景点，也是误识别的主要来源，见 2.4。

手写词典收不全是难免的，所以补了一层自动发现（src/domain_dict.py）。它扫一遍分词结果，遇到“修饰语 + 中心词”（非洲 + 草原）或“景点 + 设施”（天鹅湖 + 缆车）这种相邻写法就先合成，合成后在全文出现两次以上才收进词典。只出现一次的不收，噪声太多（“从鲸鲨馆”“看看天鹅”

也会被合成）。这一轮收进来 38 个，其中真正新增的有非洲草原、东非草原、天鹅湖缆车、海豚岛、海象岛等 11 个，其余本来就在词典里。

时间用 7 条正则，分别对应时刻（10:00）、中文时刻（10 点 30 分）、时长（30 分钟）、时段（上午/傍晚）、天序（第二天）、英文天序（Day 2）、日期（10 月 3 日）。每类记一个 subtype，便于项目 3 区分“原文写死的时刻”和“按时段推断的时段”。

2.3 实体输出与出现位置

输出 entities_v1.json 与 entities_v2.json，每条实体五个字段：type、text、para_id、start、end，时间实体多一个 subtype。start 与 end 是全篇字符位置，区间左闭右开。

```
{"doc_id": "JZG-OFF-01", "scenic_area": "九寨沟风景名胜区",
 "entity_count": 153,
 "entities": [
   {"type": "POI", "text": "原始森林", "para_id": 5, "start": 81, "end": 85},
   {"type": "TRANSPORT", "text": "观光车", "para_id": 5, "start": 86, "end": 89},
   {"type": "TIME", "text": "07:00", "para_id": 5, "start": 91, "end": 96,
    "subtype": "clock"}
 ]}
```

三类实体的数量见表 2-2。景点出现次数最多，v2 里平均每千字 18.1 次；交通与时间两类数量接近，都在 210 次上下。九寨沟官方路线那篇密度最高，2270 字里 153 个景点实体，因为它本身就是一条 46 步的路线，每步点名两个地点。

表 2-2　两版实体识别结果对比

版本	景点出现次数	不同景点词	交通次数	时间次数	合计
v1（基线）	1230	207	213	216	1659
v2（加停用词 + 合成词）	837	195	213	216	1266

2.4 人工抽查与错误分析

随机抽 18 个景点实体回原文核对，3 个不是地点：“中九寨”“守宫”“壕沟”，三例的原文与成因见表 2-3。

表 2-3　三例误识别及其成因

实体	原文片段	成因
中九寨	雨中九寨的美，是那么清澈无暇	jieba 把“雨中九寨”切成一块，以“寨”结尾被后缀规则收录
守宫	日行守宫，也就是壁虎类的	“守宫”是一种壁虎，以“宫”结尾被当成宫庙类地点
壕沟	中间隔着深深的壕沟	普通名词，以“沟”结尾，与“树正沟”同后缀

三个错误同一个原因：后缀规则只判断“以地点后缀结尾”，没判断是不是专名。中文里“寨”“宫”“沟”既是地名用字，也是普通词用字。要压掉这类误识别，后缀候选至少还要过两道检查：前面六字内出现“到/去/在/从/游览/参观”这类位移或观看动词才收，否则不收；再把项目 3 解析出来的景点序列反哺进词典，路线里出现过的名字一定是景点。这两条都能落地，这一轮没做，误识别还在。

漏识别那边补上了：复合专名（澳洲森林、海豚岛、天鹅湖缆车）原来因为 jieba 切碎进不了词典，合成词规则接上之后都识别出来了，v2 的景点出现次数从 808 涨到 837，不同景点词从 184 涨到 195。

2.5 三类实体词频可视化与迭代

第一版直接统计词频，前六名是园区 67、酒店 67、珠海 49、动物园 40、餐厅 36、九寨沟 36（图 2-1）。其中只有“九寨沟”是景点，其余是园区泛称、住宿设施和城市名。这类词在 1230 次景点实体里占 440 次，三成六。

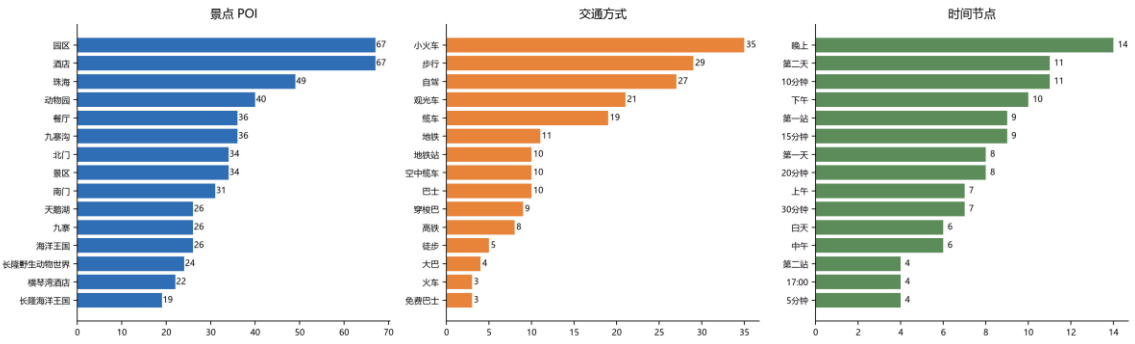


图 2-1　第一版三类实体词频（未加停用词）

按词频表整理出 47 个停用词，分三类：泛称（园区、景区、度假区、酒店、餐厅、商店）、城市与区域名（珠海、广州、九寨、四川）、动作（入园、出园、开园）。第二版做两件事：把这三类从景点实体里剔除，再加上自动发现的合成词。剔除让景点出现次数减少，合成词又补回一部分，最后从 1230 降到 837，不同景点词从 207 降到 195。交通与时间两类没有变化，说明这两项改动只作用在景点上。

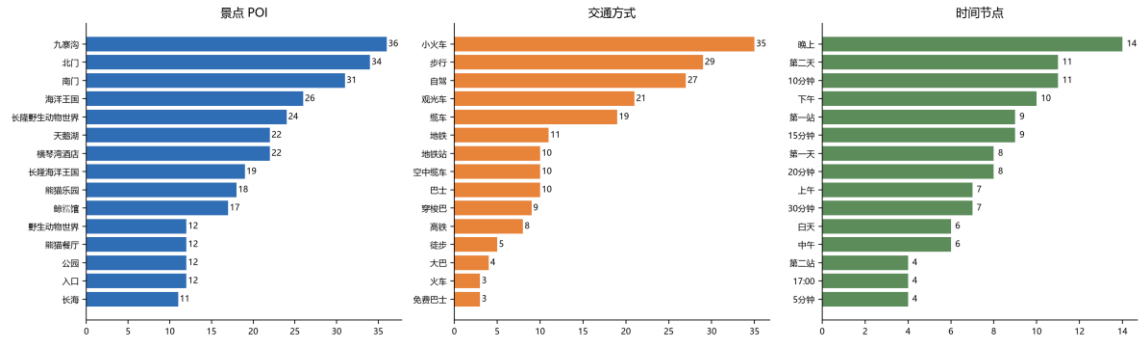


图 2-2　第二版三类实体词频（加停用词后）

第二版的词频见图 2-2，第一名变成“九寨沟”（36），后面是北门 34、南门 31、海洋王国 26、长隆野生动物世界 24、天鹅湖 22。词频表从“设施与地名”变成了“景点与出入口”，与 2.1 的定义一致。



图 2-3　第二版三类实体词云（时间类已隐藏纯钟点，避免字号被 07:00 这类词占满）

词云（图 2-3）只作形状参考：景点类集中在少数几个高频词上，长尾很长；交通类几乎被“小火车”一个词主导；时间类则以时段和时长为主，钟点过于分散，画出来只剩一堆数字，所以在词云里隐藏了。

《数据挖掘》探索性实验报告
—实验项目 3 游览路线的层级结构挖掘

3.1 路线描述模式与输入范围

官方路线共 6 篇，覆盖三种写法，另有一种零星出现。按解析难度排序见表 3-1。

表 3-1　官方路线的四种描述模式

模式	原文样子	出现在	解析难度
带时刻的编号站	第一站（10:00）：小火车游览	GZCL-OFF-01	好解析，正则一条
只有编号没有时刻	第一站：长隆欢乐世界	GZCL-OFF-03、ZHCL-OFF-01	能解析，但排不了时间
带时刻区间的箭头	沟口乘车→原始森林（07:00-08:10 一个小时车程）	JZG-OFF-01	最好解析，时间和起止点都有
纯箭头链	北门——小火车游览区——青龙山——……——出园	GZCL-OFF-01、JZG-OFF-01	只能拿到顺序，没有时间

另有“时段——地点”的写法（上午——尖叫地带）在长隆两篇里零星出现，一并处理。九寨沟三日游那部分是散文式描述（“从九寨沟县城到神仙池 58 公里，驱车前往约 1 个半小时”），没有箭头也没有编号，解析器给不出步骤，这一部分不在支持范围内。6 篇官方里 5 篇解析出了路线，剩下一篇是珠海官网发的亲子攻略长文，按“01 交通方式”“02 住宿”分节，通篇没有路线结构，没有解析。

同一篇里混用两种写法的情况要单独处理。GZCL-OFF-01 先用箭头链把整条路线概述一遍，再用“第一站（10:00）”逐站展开，两段讲的是同一趟路。两种都收会得到 26 步，同一趟走两次，步骤号从 12 跳回 1。解析器只保留逐站那一段，因为它既有序号也有时刻，箭头链留作校验。

3.2 统一结构化表示与解析程序

统一表示用 step 列表，每条 step 九个字段：序号、时段、起始时刻、结束时刻、起点、终点、活动描述、交通方式、时间来源。时间来源要区分四种：原文写死（clock）、按区间推出（range）、按时段继承（period）、还是没有（none），验证时用来区分事实与推断。

```
{"route_id": "JZG-OFF-01-R1", "format": "arrow_time+period_block",
 "steps": [
   {"step": 1, "period": "上午", "time_start": "07:00", "time_end": "08:10",
    "from": "沟口", "to": "原始森林", "activity": "", "transport": "车程",
    "time_source": "range"}
 ]}
```

解析结果 6 条路线 66 步：广州长隆三条线 11、3、3 步，九寨沟一日游 20 步、二日游 26 步，珠海长隆 3 步。九寨沟那两条是按“一日/二日/三日”标题切开的，不切会串成一条 46 步的通票。没有时刻的步骤不插值。缺时间的步骤 time_start 留空、time_source 记 none，时段能继承就继承，继承不到记“未标时段”。按平均间隔补出来的时间看着整齐，但下游会把它当成原文事实使用。

3.3 时间层级构建

时间层级按小时切四段：6–11 上午、11–13 中午、13–18 下午、18 点以后晚上。中午没有按常规的 12–14 点划，因为官方路线里 11:30 到诺日朗吃饭、13:30 已经坐车去长海，按 12–14 切会把这两步都归错。

九寨沟一日游的层级是：上午从沟口坐车到原始森林，下行经芳草海、天鹅海、箭竹海、熊猫海、五花海、镜海到诺日朗群海，8 步；中午在诺日朗中心站 2 步；下午去长海、五彩池，再回树正沟走到盆景滩出沟，10 步。广州长隆那条 11 步集中在中午和下午，上午只有小火车一步。

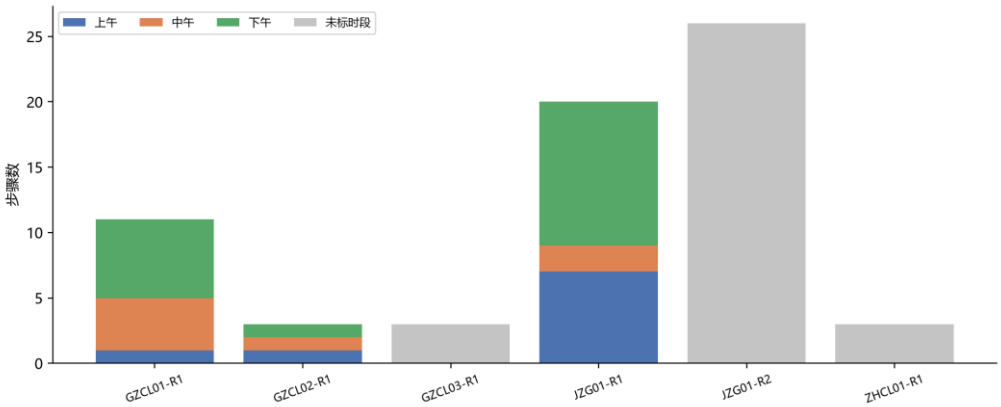


图 3-1　各条官方路线的步骤在时段上的分布

图 3-1 里三条路线落在“未标时段”：珠海那条、九寨沟二日游、广州“第一站：某园区”那条。前两条原文确实没写时刻，第三条只有编号。灰色越长说明那条路线的时间颗粒度越粗，不是解析失败。

3.4 名称归一化与稳健性

名称归一化分三层。第一层是明确的别名，手写表处理，六条：白虎山/百虎山、则渣洼沟/则查洼沟、诺日朗中心/诺日朗中心站、4D 影院/四维影院、沟口/沟口站、天鹅美食/天鹅美食广场。

第二层去动词前后缀。“沟口乘车”归一成“沟口”，“前往长海”归一成“长海”，这类词是箭头链解析时带出来的，去掉动词才是地点本身。第三层是父子关系，只标记不合并：“九寨沟”是父区域，“五花海”是子景点；“长海”和“长海观景台”是地点与观景点。

父子关系不能合并，否则覆盖率会失真。游记里只要提一次“九寨沟”，就等于覆盖了官方路线的全部景点。所以归一化表里单独有一份父区域名单，统计时排除。过度对齐的风险出现过一次。第一版没有去掉“乘车”这类动词后缀，官方序列里混进“沟口乘车”这个非景点，覆盖率从 0.47 掉到 0.40。补上动词后缀规则后回到 0.47。归一化不是越狠越好，每加一条规则都要回看指标有没有被带偏。

3.5 结构结果验证

验证做两件事：抽查单条路线每一步能否在原文找到对应句子，检查有没有漏步和顺序错乱。抽查用九寨沟一日游，它时间区间写得最全，好核对，节选六步见表 3-2。

表 3-2　九寨沟一日游逐步核对（节选 6 步）

步骤	解析结果	原文	判断
1	沟口→原始森林 07:00-08:10	沟口乘车→原始森林（07:00-08:10 一个小时车程）	对
2	原始森林→芳草海 08:10-09:00 步行	原始森林→芳草海（08:10-09:00，步行 1 小时）	对
3	芳草海→天鹅海 09:00-09:15 观光车	芳草海→天鹅海（09:00-09:15，观光车）	对
8	镜海→诺日朗群海 11:10-11:25 观光车	镜海→诺日朗群海（11:10-11:25，观光车）	对
9	诺日朗群海→诺日朗中心站 11:30-13:30	诺日朗中心站（11:30-13:30 午餐休息）	对
10	诺日朗中心站→长海 13:30-14:20 观光车	诺日朗中心站→长海（13:30-14:20，观光车）	对

20 步都能在原文找到对应句子，没有凭空生成的步骤，顺序与原文箭头方向一致。有两处原文本身前后重叠：双龙海→盆景滩写 17:00-17:20，盆景滩→沟口写 17:00-17:35，起始时刻都是 17:00，照原样记入，没有按后一段的起始时刻修正。

3.6 官方与游客实际路线对比【附加题】

对比用九寨沟一日游：官方路线 20 步 19 个地点，游客路线取自 JZG-NOTE-03 的行程总结，12 个地点。指标算五个：景点覆盖度、Jaccard、最长公共子序列、顺序一致性、时段分配，五项指标两边对比如表 3-3。

表 3-3　官方路线与游客实际路线的结构对比

指标	数值	含义
景点覆盖度	0.47（9/19）	官方 19 个地点里，游客走过 9 个
Jaccard	0.41	两边集合的交并比
最长公共子序列	6	顺序一致的最长部分
顺序一致性	0.50	LCS 除以较短序列长度
时段分配	官方 35/10/55，游客 21/7/71	上午/中午/下午所占步数比例

两边顺序对照见图 3-2，时段分配这一项差异最大。官方把 35% 的步骤放在上午，游客只有 21%；下午官方 55%、游客 71%。官方全程 600 分钟、平均每步间隔 32 分钟，游客路线原文没有时刻，这一项算不出来，只能比步数比例。

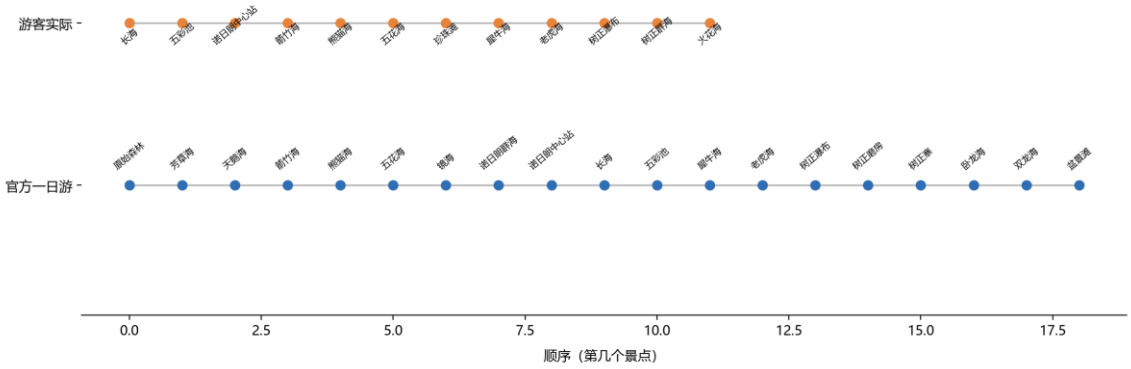


图 3-2　九寨沟一日游：官方顺序与游客实际顺序对照

差异的原因能在原文里找到依据。方向上相反：官方从沟口坐车到最里面的原始森林，一路下行；游客先到长海、五彩池，再折回树正沟，两次换乘方向相反。取舍也不同：游客跳过了原始森林、芳草海、天鹅海这三段需要步行的点，只保留珍珠滩、树正群海、火花海。节奏上，游客把 71% 的步骤压在下午，上午进沟后先坐车到终点再往回走，跟官方“上午密集、下午收尾”正好反过来。

这组对比只能说明一条路线的差别，不能推广。游客样本只有 1 篇，还是一篇结构清晰的总结，多数游记里的路线是夹在叙述里的，提取不出来。上面的三个原因属于个案解释，不是统计结论。

《数据挖掘》探索性实验报告
—实验项目 4 程序性知识的共指消解

4.1 代词检测与待消解样本

代词检测在段落的句子层面做：先用标点断句，再用正则匹配 10 个代词（它、它们、他们、她们、这里、那里、这儿、那儿、这边、那边）。命中后记录代词、所在句、上一句和全篇位置，四条信息缺一不可，“上一句”是消解时的主要线索。

限定只在游记里找。“我们”不参与，旅游文本里它多是泛指作者一行人，没有具体先行词；“其它”“其他”里的“它”先屏蔽，否则会多出一批假样本。

表 4-1　代词检测结果

代词	出现次数	主要语境
这里	57	指段落正在讲的景点，如“在这里生活着大熊猫”
它们	17	指前面出现的动物群体
它	16	指单个动物或景点
他们	6	指游客或工作人员
她们	5	指女性游客
那里	3	指远处的景点或区域

共检出 104 处（表 4-1），分布在 10 篇游记里。“这里”占 55%，比例远高于其他代词，这与游记的写法有关：介绍完一个景点，紧接着用它指代同一处。

4.2 人工标注真实先行词

从 104 处里取 13 处人工标注，每个样本标三项：先行词、类型、是否本来就没有先行词。类型分四类：POI、动物、人物、篇章（“在这里感谢”这类话语标记）。标注保存在 coref_gold.json，逐条带一句标注理由。

13 例里 3 例没有明确先行词：C001“在这里感谢那些写攻略的朋友”、C002“这里重点提醒”、C005“它们”指一类动物但上文没点名。这 3 例单独统计，不混进准确率，否则“拒答”和“答错”会被算成一回事。同一句出现两个相同代词的情况，标注时按出现顺序编号（C026-1、C026-2），避免消解时把后一个的先行词写到前一个上。这批数据里有 2 句属于这种情况，都在长隆的游记里。

4.3 消解方法与实现

消解用规则打分，不用模型。候选先行词取自代词前 60 字里的名词（以 jieba 词性 n/ns/nz/nr 等为准），再加上项目 2 识别出的 POI。同一词只保留最后一次出现的位置。

打分两项相加：词是 POI 加 3 分，距离分取 1/(距离+1)，距离按字符算，取分数最高的。加类型分是因为“这里”在后文八成指景点，而距离最近的往往是个动作或抽象名词（“智商”“策略”），纯按距离选会选错。

候选能不能生成出来，比打分更重要。三条措施都是冲着这个来的。先把项目 2 识别出的景点名回喂给 jieba（add_word），否则“黑颈天鹅”仍会被切成两块。相邻词能拼成中心词结尾的就拼一次再当候选，这条不受词频限制，只出现一次的复合专名也有机会进列表。指人的代词（他们/她们）再换一套打分，地点不加分，改给动物名加分。C004 那句“一头成年大象……他们非常聪明”，按原来那套会被前面的“亚州森林”抢走。

同分时取名字更完整的那个。“天鹅湖”和“天鹅湖缆车”分数一样时选后者，原句说的是坐缆车这件事，不是湖本身。

这一步的输出除了对应关系，还有消解后的句子。不过把“这里”换成景点名之后，句子经常更别扭（“在百虎山感谢那些朋友”），所以消解文本只作检查，不当最终产物。往项目 6 传的是“代词→先行词”的对应关系。

4.4 消解结果与输出

输出 coref_result.json，每条记录含代词、原句、候选列表（带分数与距离）、预测先行词、人工标注、是否正确。候选列表保留下来是为了区分两种错误：没找到合适候选，还是找到了但排序错了。这两种的改法完全不同，判定结果与消融对比见图 4-1。

```
{"id": "C027", "pronoun": "它", "pred": "百虎山", "gold": "百虎山",
 "candidates": [{"text": "百虎山", "is_poi": true, "dist": 3, "score": 3.25},
                {"text": "老虎", "is_poi": false, "dist": 9, "score": 0.10}],
 "correct": 1}
```

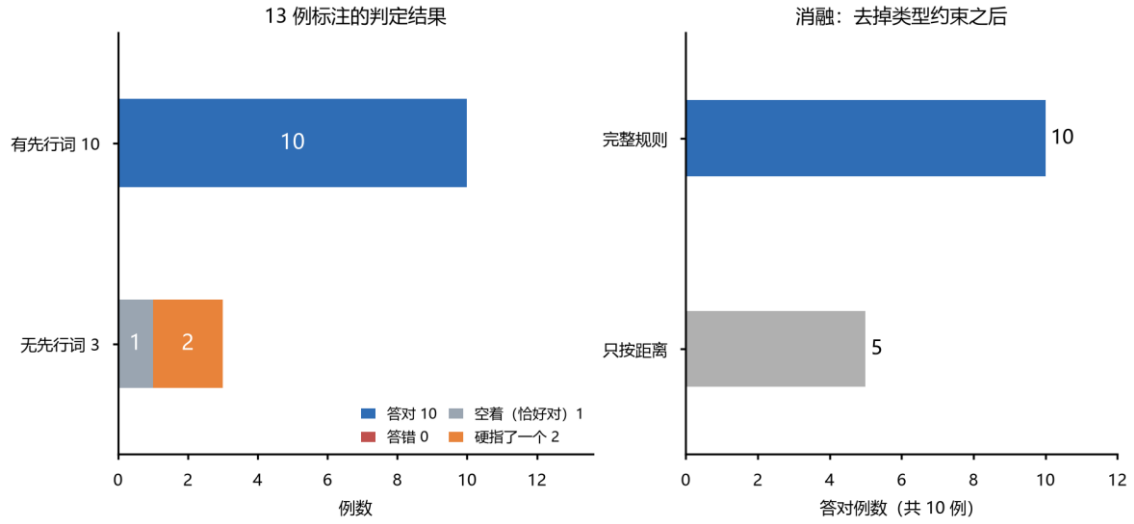


图 4-1　13 例标注的判定结果（左）与去掉类型约束后的消融（右）

4.5 人工核验与评价

13 例里 10 例有明确先行词，规则 10 例全对，准确率 1.00；覆盖率 10/13 = 0.77。另外 3 例本就没有先行词，其中 2 例被硬指了一个词，1 例恰好留空。逐例结果见表 4-2。这个 1.00 要打个折看，理由见本节最后一段。

表 4-2　10 例可判定样本的消解结果

编号	代词	预测	标注	结果
C004	他们	大象	大象	对，靠动物名约束翻盘
C006	这里	天鹅湖缆车	天鹅湖缆车	对，靠同分取长名
C009	这里	澳洲森林	澳洲森林	对，靠合成词
C010	它	黑颈天鹅	黑颈天鹅	对，靠合成词
C014	它	欧洲山地	欧洲山地	对，靠合成词
C025	那里	金蛇秘境	金蛇秘境	对
C026	这里	百虎山	百虎山	对
C027	它	百虎山	百虎山	对
C031	它	豚尾猴	豚尾猴	对
C032	这里	大熊猫中心	大熊猫中心	对

改之前是 5/10，五处错误里四处因为复合地名被切碎（“澳洲森林”→“澳洲”＋“森林”），第五处是“他们”被前面的地名抢走。加上合成词、动物名约束和同分取长名之后变成 10/10。

做了个消融：去掉全部类型约束，只在同一批候选里按距离选最近的名词，答对 5 例，准确率 0.50。类型约束在 10 例里贡献了 5 例。

这个 1.00 不能当方法的真实水平。样本只有 10 例，错一例就是十个百分点；那三条规则又都是看完标注里的错误才补的，动物名表更是照着 C004 写的，存在对着标注调规则的问题。要说明这套打分真的有用，得换一批没参与调规则的样本重测，这一步写在总结里。

《数据挖掘》探索性实验报告
—实验项目 5 条件性游览建议的抽取

5.1 输入与抽取规则设计

输入是 17 篇文档的段落文本，输出是“条件→建议”的映射表。条件句在游记里长得不统一，规则按两种句式切：有“如果/假如/若/要是”的，以第一个逗号为界，前半是条件；没有条件连词的，用建议触发词（建议、可以、记得、千万别、最好）切前后。三类规则与命中数见表 5-1。

表 5-1　三类抽取规则

规则	触发条件	切分方式	命中条数
if_pattern	条件连词：如果、假如、若、要是	连词到第一个逗号为条件，其后为建议	35
adv_pattern	建议词 + 前文 ≥4 字	建议词前为条件，前到句首或分号为界	143
adv_only	建议词前不足 4 字	整句作建议，条件留空	24

规则有优先级：先试 if_pattern，命中就不再走建议词。“如果下雨，建议带伞”如果按建议词切，条件会变成“如果下雨”，“如果”这个词反而留在条件里，读起来别扭。

5.2 批量抽取与复杂表达处理

抽之前先断句，按。！？；和换行切，长句再按逗号补切一刀。断句太粗会把两条建议粘成一条，太细会把条件和建议拆到两个句子里，最后定在分句这一级，条件句多数落在一个分句内。

两处边界问题要单独处理。“别”在“特别”“别的”“分别”里不是祈使，“可以”在“不可以”“没可以”里是被否定的，第一版就出过“不允许带别的食物”被切成“不允许带”+“可以带别的食物”这种反义结果。现在加了两道检查：看“别”前一个字是不是“特/分/区/级/类/性”，看“可以”前一个字是不是否定词。

“别有一番韵味”“别处很难见到”“另外”这类是在写景或转折，不是建议，开头就被排除。公众号文末的互动引导（关注、留言、二维码）也一并过滤，共 2 条。

切出来的条件两端还要清理一遍。切分点落在句中时，尾巴上常挂着半截话（“跳跃距离和安全距离范围专门设置了壕沟，这样我们不仅”），头上常挂着介词（“从”“在”“当”）。清理表把连词、代词、时间词逐层剥掉，剥完不足三个字就不剥。加这一层之后，抽 20 条里条件带半截话的从 7 条降到 0 条。

5.3 映射表、分类与去重

映射表字段八个：id、doc_id、scenic_area、text_type、condition、advice、rule、cond_type。cond_type 按条件里的关键词分成 11 类，分不出来记“其他”，11 类的分布见表 5-2。

表 5-2　条件类型分布（202 条）

类型	条数	典型条件
其他	125	条件为一段描述、无法归类，如“动物园九点半开园，开车族”
通用	24	没有条件的建议，如“一定要提前网上购票”
餐饮	14	自带食物、园内吃饭、餐厅价格
人群-儿童	12	带孩子、婴儿车、孩子走不动
装备	8	防晒、雨具、充电宝、现金
交通	5	停车、接驳、地铁
时间	5	早上几点到、避开高峰
票务	3	取票、身份证、年卡
人群-老人	2	带爸妈、走不动
体力	2	步行距离、爬坡
天气	2	下雨、暴晒

去重按建议文本前 24 字判断，先去完全相同，再去近重复（去掉标点后前 24 字一致）。抽取阶段 204 条，去重后 202 条。同一景区同一建议的多种说法只留一条，否则映射表会被“记得带伞”“一定要带伞”“别忘带伞”撑散。

5.4 人工校验与误差分析

随机抽 20 条人工核对，逐条看条件与建议是否配对。加了两端清理之后再抽一次同样 20 条，前后对比见表 5-3。

表 5-3　同一批 20 条抽查的前后对比

问题	例子	清理前	清理后
条件带半截话	「…设置了壕沟，这样我们不仅」→「可以看到猛兽们自由活动」	7	0
条件其实是背景陈述	「象园是一个集展示和互动于一体的园区」→「可以亲自与大象互动」	6	6
条件与建议错配	「能坐到靠窗的位置请尽量选择靠窗」→「大熊猫中心可以带小朋友们…」	1	1
抽到的不是建议	「推荐指数：※※※※」	1	1

边界问题解决掉了，剩下三类规则解决不了。背景陈述那 6 条最典型：“象园是一个集展示和互动于一体的园区”是介绍，不是条件，但句法上和建议句没有区别，规则只看词序，看不出主语是谁。要分清得用语义角色标注（LTP、HanLP）或者句法分析，这一步没做。

5.5 分类统计与可视化

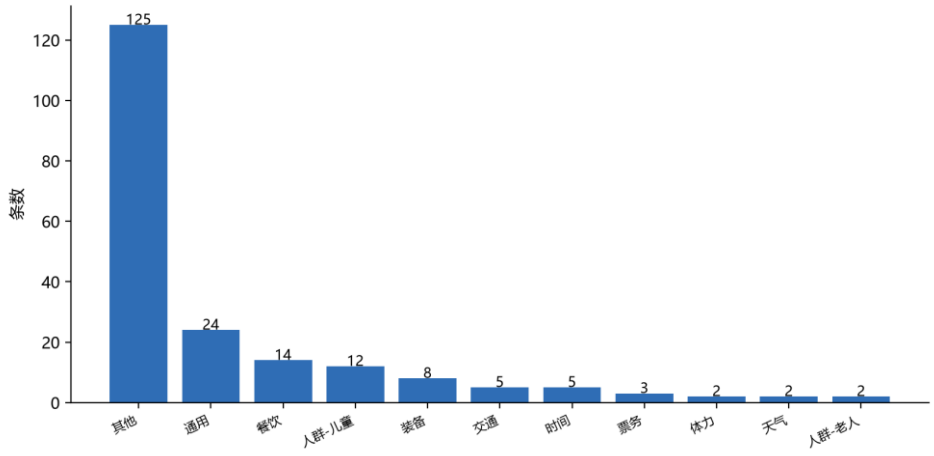


图 5-1　条件建议的类型分布

类型分布见图 5-1：202 条里 140 条来自游记、62 条来自官方路线。官方那部分多是票务和时间提示，游记那部分多是吃住行与体力安排，两类来源的分布差异明显。

《数据挖掘》探索性实验报告
—实验项目 6 多源程序性知识的融合与图谱可视化

6.1 输入关联与融合策略

输入三份：项目 3 的 routes.json（6 条路线 66 步）、项目 2 的 entities_v2.json（核对景点写法）、项目 5 的 conditions.json（作景点节点的 tips 属性）。关联键是景点名，三份都用项目 3 归一化后的写法，每份输入用到的字段见表 6-1。

表 6-1　三份输入的融合方式

输入	内容	在图里的角色	融合键
routes.json	66 步，含起止点、时刻、交通	步骤节点与 NEXT 边	to / from 字段
entities_v2.json	1266 个实体，195 个景点词	节点命名与去重依据	实体文本
conditions.json	202 条条件建议	景点节点的 tips 属性	scenic_area + 景点名

条件建议没有建成节点，挂在景点属性上。建成节点的话，一个景区要加进上百个建议节点，图会从二十几个点涨到一百多个，路径全被遮住。挂属性同样能查，代价是没法直接按建议类型遍历。

6.2 对齐样例与噪声诊断

对齐前先看两侧写法差多少。项目 3 归一化后的景点名是“原始森林”“诺日朗群海”，项目 2 的实体里还有“原始森林景区”“诺日朗群海观景台”这类写法，两边能直接对上的占八成。

对不上的分两种处理。父子关系（长海 / 长海观景台）保留两个节点，加一条 BELONG 边；纯写法差异（白虎山 / 百虎山）在项目 3 的别名表里已经合并，这里直接用合并后的名字。

噪声诊断发现两类问题。范围外的景点占了大头：项目 2 一共 195 个景点词，与九寨沟一日游路线重合的只有 19 个，其余 176 个来自另外两个景区，或只在游记里提过一次，没有入图，它们连不成路径。时间节点也偏粗：项目 3 只给了时段，没给具体钟点，图里的时间节点只有 3 个。

6.3 图谱模式（Schema）设计

表 6-2　图谱 Schema

元素	类型	含义	数量
景区	节点	一个景区一个节点，作为子图入口	1
景点	节点	路线里出现的地点，含出入口	20
时间	节点	上午/中午/下午三个时段	3
BELONG	边	景点属于景区	20
AT	边	景点落在某个时段	20
NEXT	边	相邻两步的先后关系，带交通方式属性	19

只建了一个景区的图：九寨沟一日游，节点与边的类型见表 6-2。三个景区的路线结构和时间粒度都不一样，混在一张图里会出现两套 NEXT 语义，读图的人分不清哪条边属于哪条路线。NEXT 边带三个属性：step（第几步）、transport（交通方式）、period（时段）。属性能表达“从 A 到 B 是坐观光车还是走路”，但没法表达并行关系，这也是单路线建图的原因之一。

6.4 构建实现与结构化输出

构建用 NetworkX，读 routes.json 后按角色建节点和边，输出三份：graph.json（node-link 格式）、graph_nodes.csv、graph_edges.csv。CSV 给不会用 Python 的人看，JSON 给程序读。

节点 id 直接用名称，不加前缀。景点名在三个景区之间基本不重名，只有“北门”“南门”两个长隆都有，所以只建了九寨沟的图，这个冲突暂时不存在。要扩到三个景区，节点 id 需要改成“景区码:景点名”。

```
{"nodes": [{"id": "原始森林", "label": "原始森林", "ntype": "景点",
                      "tips": ["建议早上第一批进沟"]}],
 "links": [{"source": "原始森林", "target": "芳草海", "rel": "NEXT",
            "key": "next", "step": 2, "transport": "步行"}]}
```

6.5 图谱可视化

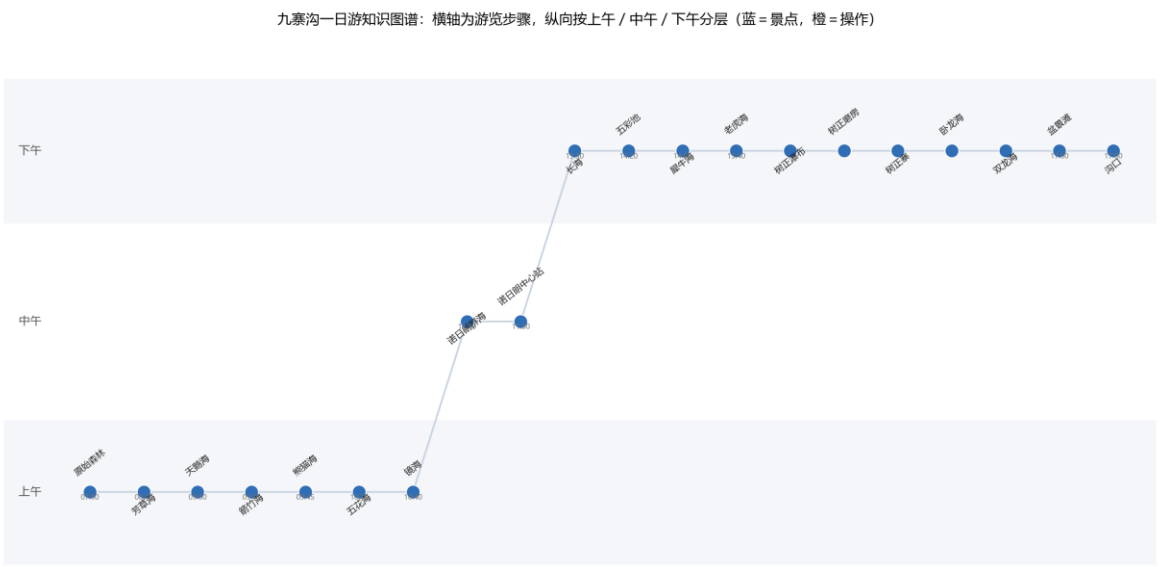


图 6-1　九寨沟一日游知识图谱：横轴为步骤，纵向按上午 / 中午 / 下午分层

图 6-1 的布局没有用弹簧模型。一条链用弹簧布局会绕成几个圈，节点叠在一起看不清。现在横轴是步骤序号，纵轴是时段，蓝点是景点，橙点是“看火烈鸟”这类动作，点上的小字是时刻。路径从左到右，一眼能看出上午密集、中午两步、下午拉长。

6.6 质量验证与结果分析

表 6-3　图谱质量检查

检查项	方法	结果
连通性	弱连通分量	1 个，20 个景点全连通
最长路径	按 NEXT 边求最长链	19 步，与路线步数一致
节点度数	统计入度出度	中间节点度为 2，首尾为 1，没有孤立点
边语义	逐条核对 transport	19 条 NEXT 边中 18 条有交通方式，1 条原文未写
属性完整性	检查 tips	2 个景点挂上了条件建议，其余 18 个没有对应建议

对照表 6-3，两处不足。时间粒度上，图里只有三个时段节点，没有具体钟点，问“几点到五花海”这种问题答不了。建议覆盖更稀疏：20 个景点里只有 2 个挂了 tips，因为项目 5 的建议大多不带景点名，只能按景区挂到父节点上。

《数据挖掘》探索性实验报告
—实验项目 7 程序性知识应用：旅游规划Agent

7.1 架构与 LLM 接入

Agent 按 ReAct 的方式做（图 7-1 左）：每一轮先想下一步做什么，再调一个工具，读回结果，根据结果决定下一轮。循环的出口有两个：信息够了就生成行程，关键信息缺了就反问用户。七个测试场景里，出口不止这两个，还有“库里没有这个景区”和“约束冲突”，见 7.3。

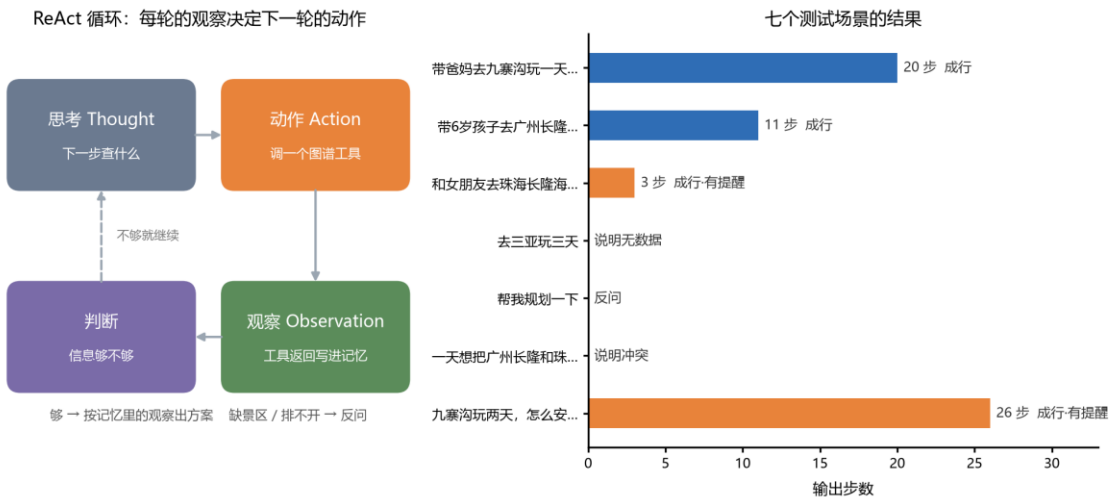


图 7-1　ReAct 循环结构（左）与七个测试场景的结果（右）

工具五个，全部读项目 3/5/6 的产物，不把图谱内容拼进提示词。check_area 查景区是否存在，get_route 取结构化路线，search_spots 取景点，get_tips 取注意事项，tailor_route 按人群裁剪并给步骤打标。每个工具有明确的入参和返回结构，见表 7-1。

“谁来想”这一步可以换。设了 OPENAI_API_KEY 就由模型输出下一步的 Action，没设就用规则策略，两条路共用同一个循环和同一套工具。本次跑的是规则策略，LLM 分支的代码在 p7_agent.py 的 _llm_think() 里，没有 key 时自动回退到规则，不会中断。

7.2 图谱查询工具封装

表 7-1　五个工具的输入输出

工具	输入	输出	数据来源
check_area	area	{exists, routes, available}	routes.json
get_route	area, days	{route_id, steps, step_count, span, coarse}	routes.json
search_spots	area, visitor, top_k	{hit, spots[]}	entities_v2.json
get_tips	area, keyword	{count, tips[]}	conditions.json
tailor_route	route, visitor	{steps[], count, skipped}	get_route 的返回

get_route 多返回一个 coarse 标志：步数少于 5 时置真，表示这条路线只到园区级。这个标志是给策略用的：用户要两天、路线只有三步，就得在回答里说明，不能当两日游方案直接输出。

工具异常不抛出。try 里跑，出错时返回 {ok: False, error}，观察文本记一条，策略继续走下一步或直接收尾。这样一次工具失败不会让整个会话崩掉。

7.3 闭环编排与回退策略

策略是一个按状态判断的函数，读槽位和已有观察决定下一步。四类判断按顺序：提了多个景区但只给一天，直接说明排不开；没提取到景区，反问；景区不在库里，说明没有；其余情况按 取路线 → 取景点 → 取建议 → 裁剪 → 收尾 逐轮推进。

```
# 规则策略（p7_agent.py 的 _rule_think），按顺序判
if len(areas) >= 2 and days <= 1:      return final(conflict)
if not area:                            return ask(景区)
if check_area not in obs:               return action(check_area)
if not obs["check_area"].exists:        return final(no_data)
if get_route not in obs:                return action(get_route)
if route.coarse and days >= 2:          note("颗粒度不足")
if search_spots not in obs:             return action(search_spots)
if get_tips not in obs:                 return action(get_tips)
if tailor_route not in obs:             return action(tailor_route)
return final(done)
```

五种收尾状态：ok 正常出方案；partial 出方案但带提醒；need_info 反问；no_data 库里没有；conflict 约束冲突。状态写进 agent_runs.json，测试时按状态统计，不看回答文本是否“像样”。

回退有三处。路线颗粒度不足时加提醒而不是换路线，因为库里没有更细的路线可换；工具返回空时记 note 并继续，不重试同一个调用，避免循环空转；轮次用完但已经拿到路线时，直接按原路线输出，并注明没做人群裁剪。

7.4 多场景功能与鲁棒性测试

七个场景覆盖四类约束：人群（老人、亲子、情侣）、天数（一天、两天）、景区（三个都试过）、异常输入（不存在的地方、信息不足、约束冲突）。结果见表 7-2。

表 7-2　七个场景的测试结果

输入	预期	实际结果	状态
带爸妈去九寨沟一天，走不动太久	识别老人，步行段标注	20 步，8 处步行标“可跳过”	ok
带 6 岁孩子去广州长隆一天	识别亲子，给完整步骤	11 步，3 处标“适合孩子”	ok
情侣去珠海长隆两天	识别情侣与两天	3 步园区级路线，提示撑不起两天	partial
九寨沟玩两天	取二日游路线	26 步，提示原文没写时刻	partial
去三亚玩三天	如实说没有	回“没有三亚的数据”，列出可用的三个景区	no_data
帮我规划一下	反问缺什么	反问去哪个景区	need_info
一天玩广州长隆和珠海长隆	说明排不开	说明两地不在同一片区域，建议分两天	conflict

四个场景成行（含两个带提醒），三个场景给出了非行程回答。信息不足时反问缺什么，约束冲突时说明两地排不开，这两种情况都单独处理，不并到“没有数据”里。

只有两个场景真正用到了图谱的细粒度信息：九寨沟和广州长隆。珠海那条路线只有三步，不管用户说几天，输出都是三个园区名，这说明问题不在 Agent，在项目 3 对这篇官网文档的解析。

7.5 一次完整执行与失败分析

拿第一个场景完整走一遍，七轮，每轮的思考、动作、观察都记在 agent_runs.json 里，逐轮结果见表 7-3。

表 7-3　九寨沟一日游的 ReAct 轮次

轮次	思考（为什么做这一步）	动作	观察
0	先拆约束	parse_slots	九寨沟 / 老人 / 1 天
1	确认景区在不在库里	check_area	库里有 2 条路线
2	取符合天数的路线	get_route	JZG-OFF-01-R1，20 步，07:00—17:35
3	看有哪些景点可排	search_spots	命中 68 个，取前 8 个
4	取注意事项	get_tips	5 条
5	按人群裁一版	tailor_route	成行 20 步，8 处标“可跳过”
6	信息够了	final	—

核对方式是把回答里的每个景点名回查 routes.json：20 个景点都能在 JZG-OFF-01-R1 的步骤里找到，时刻也对得上（07:00、08:10、09:00……17:00）。回答里没有原文没有的景点，说明内容来自工具返回，不是模型自由发挥。

输出里有两处不太合理。“可跳过”标了 8 处，几乎每两步一个，对老人来说提示太密，因为裁剪规则只看交通方式是不是步行，没问这段路多长：九寨沟的步行段落多在 1 公里以内，真正走不动的只有原始森林到芳草海那一段。出发时间也是，07:00 对老人偏早，但官方路线就是这么写的，Agent 照搬了，没做合理性调整。

还有一处是数据问题：二日游那条 26 步的路线原文没有时刻，输出里全是地点名，看不出每天几点到哪。Agent 在提醒里说明了这一点，但没有能力补时间。

7.6 交互入口与配置

入口是命令行，两种用法：python src/p7_agent.py 跑七个预设场景，python src/p7_agent.py --ask "带老人去九寨沟一天" 跑单条。加 --llm 强制走模型决策，三个环境变量见表 7-4。

表 7-4 运行配置

配置项	作用	默认值
OPENAI_API_KEY	模型 key，不设则走规则策略	空
OPENAI_BASE_URL	接口地址，兼容其他 OpenAI 格式服务	https://api.openai.com/v1
OPENAI_MODEL	模型名	gpt-4o-mini
--max_turns	循环上限，超过就收尾	6

每次运行输出三份：agent_runs.json（逐轮记录与结果）、agent_demo.txt（可读的对话记录）、figures/p7_react.png（循环结构与场景结果）。复现只需 data/processed 下的四份中间产物，不需要联网。

7.7 旅游规划 Agent 示范用例

完整演示用第一个场景，输入“带爸妈去九寨沟玩一天，他们走不动太久”。

```
问：带爸妈去九寨沟玩一天，他们走不动太久


[0] parse_slots  → 九寨沟 / 老人 / 1 天
[1] check_area   → 库里有 2 条路线
[2] get_route    → JZG-OFF-01-R1，20 步，07:00–17:35
[3] search_spots → 命中 68 个，取前 8 个
[4] get_tips     → 5 条
[5] tailor_route → 成行 20 步，8 处标“可跳过”
[6] final        → 输出行程


答：按九寨沟的一日精华游给你排了 20 步：
【上午】
  原始森林（07:00）
  芳草海（08:10），步行（可跳过）
  天鹅海（09:00），观光车
  ……
【下午】
  盆景滩（17:00），观光车
```

注意：一定要提前网上购票，当天不出票；建议租婴儿车。

七轮里有五轮在调工具，第六轮才生成回答。回答里的地点、时刻、交通方式全部来自 get_route 的返回，注意事项来自 get_tips，“可跳过”标记来自 tailor_route，没有任何一项是模型自己补的。

《数据挖掘》探索性实验报告
—实验总结：探索性实验总结反思

一、回顾并记录实验过程中出现的至少 2 个失败现象，并深入分析可能导致这些问题的原因；尝试提出针对性的改进思路或优化方案

1、失败现象记录

后缀规则把普通名词收成了景点。抽 18 个景点实体回原文核对，“中九寨”“守宫”“壕沟”三个不是地点。“中九寨”来自“雨中九寨”，“守宫”是一种壁虎，“壕沟”是动物园的隔离沟。这一类到现在还在，没有修掉。条件抽取里 125 条落在“其他”，占 62%。这部分不是切错，是条件本身就是一句背景介绍，比如“象园是一个集展示和互动于一体的园区”。规则看词序，分不出“介绍”和“前提”。

珠海长隆要“两天”，Agent 只输出三步。那篇官网文档只有三个园区名，没有顺序也没有时刻，项目 3 只能解析出三步，Agent 拿到三步也只能输出三步。共指的 1.00 是调出来的。先看标注里的错，再补规则，合成词、动物名表、同分取长名，三条都是这么来的。10 个样本上全对，但换一批没参与调规则的样本还能不能全对，不知道。

2、失败原因深度分析

四个现象可以归到三层。

最底下的一层是形式与语义的错位：后缀误判和条件落在“其他”都由它管着。后缀规则和条件切分都只看词形和词序，“以寨结尾”和“是地名”不是一回事，“分句在前”和“是条件”也不是一回事。规则能做形式判断，做不了语义判断，这是方法的固有边界，不是参数没调好。

再往上是数据粒度，珠海只输出三步就是这一层的事。官网那篇文档 1254 字只写了三个园区名，下游所有项目都建立在这个粒度上：路线三步、图谱只敢画九寨沟、Agent 排不出两天。上游没有的东西下游补不出来。

最上面一层是评估本身，共指的 1.00 卡在这里。标注 13 例、条件抽查 20 条，样本这么小，一次误判就是十个百分点，规则又都是照着这些样本调的，两个问题叠在一起，报出来的数只能说明方向。

分词切碎曾经是最大的问题（改之前共指错五例里的四例），合成词规则接上之后缓解了。但它现在换了个形式继续存在：项目 2 的词典仍然收不全，漏识别的比例没有量过。

3、改进/优化方案

针对后缀误判，可以在现有规则上加两道过滤，成本不高：候选词前面六字内出现过“到/去/在/从/游览/参观”这类位移或观看动词才收；再把项目 3 解析出来的景点序列反哺进词典，路线里出现过的名字一定是景点。这两条能压掉“中九寨”“壕沟”，对“守宫”这种真动物名仍然无效，那种情况需要词表干预。

针对背景陈述被当成条件，规则已经到头了，得换工具。用 LTP 或 HanLP 做语义角色标注，把“象园是……”这种判断句和“开车族一定要早点去”这种条件句分开，判断依据是主语的语义角色而不是词序。这一步没做，工作量超出本轮范围。针对对着标注调规则的问题，补一批不参与调参的样本。做法是把 104 处代词里还没标的 91 处再标 40 处，条件再抽 40 条，全部标完之后冻结规则，然后只在新样本上跑一次。这个数才是能写进结论的数。

针对路线颗粒度，在采集阶段就判断文档能不能用。现在只看能不能切出步骤，不看够不够用。加一条“路线步数 / 天数 ≥ 5”的检查，不达标的标成“仅园区级”，Agent 拿到这个标记直接提示用户，不用等到输出后才发现。

1、梳理现有技术方案的局限性

数据规模小。17 篇、4.6 万字、三个景区，其中珠海只有 5 篇、九寨沟只有 1 条官方路线。用户路线对比只有 1 条样本，条件分类里 62% 落在“其他”，这两个数字都撑不起统计结论。

评估样本更小。共指标了 13 例，条件抽了 20 条，实体抽了 18 个。规则又是照着这些样本调的，所以报告里那几个准确率只能当方向看。规则泛化能力弱。实体词典按景区手写，条件触发词表也是人工列的，换一个景区要重写一遍；合成词规则依赖一份中心词表，中心词表没覆盖的类型照样漏。

图谱太浅。24 个节点、59 条边，只有九寨沟一条路线的骨架，没有景点属性（开放时间、适合人群、排队时长），也没有替代路线。20 个景点里只有 2 个挂上了条件建议。

LLM 没有实测。代码留了接口，环境里没有 key，这轮跑的全是规则策略。规则在七个场景里表现稳定,但它处理不了没见过的表达，“想在九寨沟看日出几点进沟合适”会被解析成“九寨沟 +1 天”，答非所问。

2、选定一个优先突破的技术点，并给出说明

如果有一个学期,优先突破的是领域分词与实体边界。四个项目里三个的错误都从这里来:项目 2 的后缀误判、项目 4 的四例切碎、项目 5 的条件边界。这一层不解决，上面的规则、模型、Agent 都建在不稳的地基上。

选它而不是选 LLM 接入，理由是可验证。LLM 能让回答更顺，但准确率提升说不清来自模型还是来自数据；分词改进可以直接用同一套标注测出来，改一步看一步。

3、突破该技术点的研究思路或实施方案

想法是先拿现有产物自动造词典。项目 3 已经解析出 6 条路线 66 步，里面的景点名逐个核对过；项目 2 的 837 个景点实体里，跨文档出现两次以上的可靠度也高。两部分合并去重，能得到两三百词的领域词典，直接喂给 jieba.add_word。

词典加进去还得带词性约束。现在只加词、不加词性，jieba 仍可能把它切开再拼回去。把景点词标成 ns（地名）、设施标成 n，切分时优先成词，可以解决一部分“黑颈天鹅”式的问题。之后要有一版带边界标注的小评估集。现在只有 13 例共指标注和 20 条条件抽查，不足以判断改动有没有效。标 200 条实体边界（起止位置），每次改词典都跑一遍，看精确率和召回率怎么变，而不是靠眼看。

接 LLM 排在最后。等词典和评估集稳定了，用 LLM 做两件规则做不好的事：判断一个没见过的短语是不是景点，以及把“走不动太久”这类模糊约束翻译成可执行的过滤条件。这两件事规则写不出来，LLM 做起来成本低，也正好有评估集可以验收。

附录

一、提交前自查

☐ 封面及小组贡献页已填写，组内评分合计为 N×100%。
☐ 7 个实验及总结反思内容完整；未实现 / 未选做部分如实说明。
☐ 报告、代码、结构化输出、图表和日志相互对应。
☐ 总结含至少 2 个失败现象、至少 2 条具体局限及仅 1 个研究突破点。
☐ 排版规范、语句通顺、图表清晰（图有图题，表有表题，图按顺序标号、图按顺序标号）。

二、提交时间与方式（按任务书）

截止：2026 年 11 月 20 日 24:00 前。由各小组组长提交

文档命名方式：第 X 组_组长姓名_《数据挖掘》探索性实验报告

蔡毅老师班：1099235614@qq.com
许可老师班：2118151758@qq.com